



## Lab Manual – 4

Code/Course: 23CSE312/Distributed Systems

**Topic:** Message Passing Interface (MPI) on a localhost

**[CO05]** [BTL-3]

**Student Name:** G. Tejaswini

**Roll No.:** AM.SC.U4CSE23219

**Batch/Section:** CSE-C

**Date:** 27/02/2026

## 1 Introduction

Message Passing Interface (MPI) is a standardized and portable communication protocol designed for parallel and distributed computing. It enables multiple processes to communicate with each other by explicitly sending and receiving messages. MPI is widely used in high-performance computing (HPC) environments such as clusters and supercomputers. MPI follows a message-passing model, where processes execute in separate memory spaces and exchange data through communication primitives. It supports both point-to-point communication (such as `send()` and `recv()`) and collective communication (such as broadcast, scatter, and gather operations). MPI itself is a **standard specification**, not a software program.

## 2 MPICH

MPICH is an open-source implementation of the MPI standard. It provides the actual software library that allows MPI-based programs to execute on various platforms, including Linux and Windows. MPICH translates MPI function calls (such as MPI Send and MPI Recv) into low-level communication operations managed by the operating system and network infrastructure. It is widely recognized for its portability, efficiency, and strict compliance with MPI standards.

In this experiment, MPI is implemented on a single machine (localhost) using Python and the `mpi4py` library. Two processes simulate distributed communication by exchanging simple “ping” and “pong” messages. Even though the processes execute on the same physical machine, they behave as independent distributed processes.

## 3 Aim

To implement interprocess communication using MPI on localhost and demonstrate basic message passing between two processes using Python.

## 4 Objectives

- To understand the fundamentals of Message Passing Interface (MPI).
- To install and configure an MPI implementation on the system.

- To use the mpi4py library for message passing in Python.
- To simulate communication between two independent processes.
- To execute multiple MPI processes using mpirun or mpiexec.

## 5 System Requirements

- Operating System:
  - Linux (Ubuntu/Debian recommended)
  - Windows (using MS-MPI or MPICH)
- Python 3
- MPI Implementation:
  - Linux: MPICH
  - Windows: Microsoft MPI (MS-MPI) or MPICH
- Python Library: mpi4py
- Command-line access for executing MPI programs

## 6 Installation Steps

### For Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install mpich
pip install mpi4py
```

## 7 MPI Setup and Execution on Windows (Localhost)

### For Windows major steps

- Download and install Microsoft MPI (MS-MPI) or MPICH.
- Ensure MPI binaries are added to the system PATH.
- Install mpi4py:

### 1. Install an MPI Implementation for Windows

#### Option A: Microsoft MPI (MS-MPI)

- Download from:  
<https://docs.microsoft.com/en-us/message-passing-interface/microsoft-mpi>
- Run MSMpiSetup.exe and follow the installation instructions.

#### Option B: MPICH for Windows

- Download from: <https://www.mpich.org/downloads/>
- Run the installer and accept the default configuration.

## 2. Add MPI to the System PATH

- Open Start → Settings → System → About → Advanced system settings.
- Click on Environment Variables.
- Under System variables, select Path and click Edit.
- Add the MPI binary path. For example:
  - MS-MPI: C:\Program Files\Microsoft MPI\Bin\
  - MPICH: C:\Program Files\MPICH\bin\
- Click OK to save the changes.

## 3. Install Python and mpi4py

- Install Python from: <https://python.org>
- Open PowerShell or Command Prompt and execute:

```
pip install mpi4py
```

```
pip install mpi4py
```

## 8 Program Implementation

Create a Python script (for example, pingpong.py):

**Note:** You can split the code below into two files **ping.py** and **pong.py**

```
from mpi4py import MPI
```

```
comm = MPI.COMM_WORLD  
rank = comm.Get_rank()  
size = comm.Get_size()
```

```
if size < 2:  
    print("Run the program with at least 2 processes.")  
    exit()
```

```
if rank == 0:  
    print("Process 0: Sending 'ping' to Process 1")  
    comm.send("ping", dest=1)  
    message = comm.recv(source=1)  
    print(f"Process 0: Received message from Process 1: {message}")
```

```
elif rank == 1:  
    message = comm.recv(source=0)  
    print(f"Process 1: Received message from Process 0: {message}")  
    comm.send("pong", dest=0)  
    print("Process 1: Sent 'pong' to Process 0")
```

## 9 Execution Steps

Run the program using:

```
mpirun -n 2 python pingpong.py
```

Here:

- -n 2 specifies that two MPI processes should be created.
- Both processes execute the same script.
- Each process determines its role using rank.

## 10 Expected Output

Process 0: Sending 'ping' to Process 1

Process 1: Received message from Process 0: ping

Process 1: Sent 'pong' to Process 0

Process 0: Received message from Process 1: pong

## 11 Explanation

- MPI.COMM\_WORLD represents the global communicator containing all processes.
- Get\_rank() returns the unique identifier of each process.
- Get\_size() returns the total number of processes.
- send() and recv() are blocking communication calls.
- Even though both processes run on the same machine, MPI treats them as separate distributed processes.

## 12 Submission Requirements

- Source code file (pingpong.py).
- Screenshots of successful execution.
- Explanation of message flow between the processes.

## Results

```
PS C:\Users\LENOVO> cd desktop
PS C:\Users\LENOVO\desktop> cd MPI_LAB
PS C:\Users\LENOVO\desktop\MPI_LAB> mpiexec -n 2 python pingpong.py
Process 1: Received message from Process 0: ping
Process 1: Sent 'pong' to Process 0
Process 0: Sending 'ping' to Process 1
Process 0: Received message from Process 1: pong
PS C:\Users\LENOVO\desktop\MPI_LAB>
```

## Message Flow Explanation

### 1. Program Start

- When the program begins, all running processes join a common group called **MPI.COMM\_WORLD**.
- Each process gets a unique ID number called a **rank** using `Get_rank()`.  
In this program, there are two processes: **rank 0 and rank 1**.
- The total number of running processes is checked using `Get_size()`, which shows that two processes are active.

### 2. Communication Starts (Rank 0)

- The process with **rank 0** begins the communication.
- It creates a message containing the word **"ping"**.
- Using the `send()` function, it sends this message to **rank 1**.
- After sending, it waits for a reply.

### 3. Reply from Rank 1

- The process with **rank 1** waits to receive a message from rank 0.
- It receives the message using the `recv()` function.
- After getting **"ping"**, it prepares a reply message **"pong"**.
- Then it sends the reply back to rank 0.

### 4. Final Receiving

- Rank 0 receives the **"pong"** message from rank 1.
- This confirms that both processes communicated successfully.

### 5. Overall Communication

- The program shows direct communication between two processes.
- Both `send()` and `recv()` are blocking operations, meaning the program waits until the data transfer is complete.
- Even though both processes run on the same computer, MPI treats them as separate parallel processes.